تمرین هفته سوم: سوپرمارکت با npm و TypeScript

در هفته دوم صفحه سوپرمارکت شما با `fetch` داده‌ها را از API می‌خواند و با JavaScript کارت‌ها را می‌ساخت. همه کدها در یک فایل `script.js` بود و هیچ ابزاری بررسی نمی‌کرد که مثلاً `querySelector` واقعاً چیزی پیدا کرده یا نه.

در این تمرین پوشه سوپرمارکت خود را به یک **پروژه واقعی** تبدیل می‌کنید:

- پوشه `shops/<slug>/` شما یک پروژه npm می‌شود (`package.json`).
- TypeScript و Vite نصب می‌شوند.
- فایل `script.js` به `script.ts` تبدیل می‌شود و متغیرهای آن نوع (Type) می‌گیرند.
- دو پکیج واقعی از npm پیدا، نصب و در صفحه استفاده می‌کنید.

این تمرین دو سطح دارد:

- **سطح قبولی (حداقل تحویل):** پروژه npm، تبدیل به `script.ts`، نوع‌دار کردن همه متغیرهای DOM و استفاده از دو پکیج npm
- **امتیاز اضافه:** نوع‌دار کردن **همه** متغیرها، پارامترها و داده‌های API؛ بدون هیچ `any`

این تمرین درس‌های هفته سوم را کنار هم قرار می‌دهد:

- Node، `package.json` و اسکریپت‌ها
- npm، نسخه‌ها و `package-lock.json`
- Vite: `dev`، `build` و `preview`
- TypeScript: نوع‌های Union، `type`، DOM و `strict`

قبل از شروع

- نسخه Node را بررسی کنید: `node -v`. برای Vite به **Node 22** یا جدیدتر نیاز دارید.
- **همه دستورهای این تمرین داخل پوشه سوپرمارکت خودتان اجرا می‌شوند** (`cd shops/<slug>`)، نه در ریشه مخزن.
- اگر فایل JavaScript شما نام دیگری دارد (مثلاً `app.js`)، هر جا در این متن `script.js` نوشته شده، نام فایل خودتان را بگذارید.

ساختار پوشه شما در پایان تمرین:

```
shops/<slug>/
  index.html
  style.css
  script.ts          ← قبلاً script.js
  package.json       ← npm init
  package-lock.json  ← npm install (حتماً Commit شود)
  tsconfig.json
  vite.config.ts
  .gitignore         ← dist/
  node_modules/      ← هرگز Commit نشود
  dist/              ← خروجی npm run build، هرگز Commit نشود
```

گام ۱: پروژه npm

```
npm init -y
npm install -D typescript vite
```

سپس `package.json` را باز کنید و آن را شبیه این کنید:

```
{
  "name": "kiss",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc --noEmit && vite build",
    "preview": "vite preview",
    "check": "tsc --noEmit"
  },
  "devDependencies": {
    "typescript": "^7.0.2",
    "vite": "^8.3.4"
  }
}
```

راهنمایی:

- `npm init -y` مقدار `"type": "commonjs"` را می‌نویسد. آن را به `"module"` تغییر دهید تا `import` / `export` کار کند.
- `"private": true` جلوی انتشار تصادفی پروژه روی npm را می‌گیرد.
- TypeScript و Vite ابزار ساختن هستند، نه کدی که به مرورگر کاربر می‌رود؛ برای همین با `-D` در `devDependencies` قرار می‌گیرند.
- اسکریپت `build` اول `tsc` را اجرا می‌کند. اگر TypeScript خطا پیدا کند، `&&` اجازه نمی‌دهد Build ساخته شود.

یک فایل `.gitignore` هم در پوشه خودتان بسازید:

```
dist/
```

`node_modules/` از قبل در `.gitignore` ریشه مخزن هست. فایل `package-lock.json` را **حتماً** Commit کنید.

گام ۲: تنظیم TypeScript و Vite

tsconfig.json را در پوشه خودتان بسازید:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "types": ["vite/client"],
    "strict": true,
    "noImplicitAny": false,
    "noEmit": true,
    "skipLibCheck": true
  },
  "include": ["*.ts"]
}
```

راهنمایی: خط `"noImplicitAny": false` مخصوص سطح قبولی است. برای امتیاز اضافه این خط را پاک می‌کنید.

vite.config.ts را هم بسازید:

```
import { defineConfig } from 'vite'

// ../../api/products.json becomes /api/products.json on the dev server,
// so send /api to the real Gang of Forty site.
const api = { '/api': { target: 'https://gof.zzb.cx', changeOrigin: true } }

export default defineConfig({
  base: './',
  server: { proxy: api },
  preview: { proxy: api },
})
```

چرا این فایل لازم است؟ Vite پوشه `shops/<slug>/` را مثل ریشه یک سایت اجرا می‌کند. آدرس `../../api/products.json` در هفته دوم درست کار می‌کرد، اما زیر Vite به `/api/products.json` تبدیل می‌شود. Vite چنین فایلی ندارد و به‌جای JSON، صفحه `index.html` را برمی‌گرداند. در نتیجه `res.json()` خطا می‌دهد، با اینکه `res.ok` برابر `true` است. این تنظیم درخواست‌های `/api` را به سایت واقعی Gang of Forty می‌فرستد. لازم نیست کد `fetch` خود را تغییر دهید.

گام ۳: از script.js به script.ts

۱. فایل را تغییر نام دهید: `git mv script.js script.ts`
۲. در `index.html` تگ اسکریپت را عوض کنید:

```
<script type="module" src="./script.ts"></script>
```

۱. صفحه را اجرا کنید:

```
npm run dev     # آدرس را که چاپ می‌کند باز کنید
npm run check   # TypeScript چه خطاهایی پیدا می‌کند؟
```

راهنمایی: دیگر صفحه را با Live Server یا دابل‌کلیک باز نکنید. مرورگر فایل `.ts` را نمی‌فهمد؛ فقط `npm run dev` آن را برای مرورگر به JavaScript تبدیل می‌کند.

خروجی واقعی `npm run check` روی یک صفحه نمونه هفته دوم:

```
script.ts(2,7): error TS2451: Cannot redeclare block-scoped variable 'status'.
script.ts(5,1): error TS18047: 'cartCount' is possibly 'null'.
script.ts(24,7): error TS18047: 'list' is possibly 'null'.
script.ts(26,12): error TS2339: Property 'textContent' does not exist on type
'string'.
script.ts(34,27): error TS2339: Property 'closest' does not exist on type
'EventTarget'.
script.ts(37,37): error TS2345: Argument of type 'number' is not assignable to
parameter of type 'string'.
```

هر خطا را بخوانید: نام فایل، شماره خط، چه نوعی انتظار می‌رفت و چه نوعی رسید. هدف سطح قبولی این است که این فهرست **خالی** شود.

سطح قبولی: DOM نوع‌دار

همه متغیرهایی که یک عنصر DOM نگه می‌دارند باید نوع دقیق داشته باشند.

- **querySelector با نوع:** به‌جای Element کلی، نوع دقیق را بخواهید:

```
const list = document.querySelector<HTMLUListElement>('#dairy')
const search = document.querySelector<HTMLInputElement>('#search')
const status = document.querySelector<HTMLParagraphElement>('#status')
const cartCount = document.querySelector<HTMLElement>('#cart-count')
if (!list || !search || !status || !cartCount) throw new Error('index.html is missing
an element')
```

- **عنصرهایی که می‌سازید:**

```
const li: HTMLLIElement = document.createElement('li')
const add: HTMLButtonElement = document.createElement('button')
```

- **رویدادها:** نوع پارامتر رویداد را بنویسید و قبل از closest بررسی کنید که e.target واقعاً یک Element است:

```
list.addEventListener('click', (e: MouseEvent) => {
  if (!(e.target instanceof Element)) return
  const button = e.target.closest<HTMLButtonElement>('button')
  if (!button) return
  // …
})
```

نوع‌های رایج DOM:

- HTMLLIElement → ، HTMLUListElement →
- HTMLFormElement → <form>، HTMLInputElement → <input>، HTMLButtonElement → <button>
- HTMLTemplateElement → <template>، HTMLImageElement → <img>، HTMLAnchorElement → <a>
- HTMLElement → و <b>، ، <section>
- رویدادها: MouseEvent (کلیک)، KeyboardEvent، SubmitEvent و Event (برای input)

سطح قبولی: خطاهای رایج و راه‌حل

این‌ها خطاهای واقعی‌ای هستند که هنگام تبدیل صفحه هفته دوم می‌بینید:

- **'Cannot redeclare block-scoped variable 'status':** فایلی که هیچ `import` ندارد، یک اسکریپت سراسری حساب می‌شود و `status` با `window.status` مرورگر تداخل دارد. وقتی پکیج‌ها را `import` کنید (پایین‌تر)، این خطا خودبه‌خود حل می‌شود.
- **'list' is possibly 'null':** `querySelector` شاید چیزی پیدا نکند. یک بار بالای فایل با `if (!list) …` بررسی کنید.
- **'possibly 'null' داخل تابع، با اینکه بالای فایل بررسی کرده‌اید:** TypeScript این بررسی را داخل `function` معمولی به خاطر نمی‌آورد، چون آن تابع Hoist می‌شود. تابع را به شکل Arrow و با `const` بنویسید: `const loadProducts = async () => { … }`
- **'Argument of type 'number' is not assignable to parameter of type 'string':** `localStorage.setItem` و `dataset` فقط رشته می‌گیرند: `String(cart)`
- **'Property 'textContent' does not exist on type 'string':** نام یک متغیر را دو بار برای دو چیز متفاوت استفاده کرده‌اید (مثلاً `name` هم رشته است و هم عنصر). نام‌ها را جدا کنید.

از `!` و `as` برای ساکت کردن خطا استفاده نکنید. علت خطا را درست کنید.

سطح قبولی: دو پکیج از npm

حداقل **دو** پکیج از npmjs.com پیدا کنید، در پوشه خودتان نصب کنید و در صفحه **به‌درستی** استفاده کنید.

```
npm install <package-name>
```

قوانین انتخاب پکیج:

- بدون `-D` نصب شود: در بخش `dependencies` قرار بگیرد، چون کد آن به مرورگر کاربر می‌رسد.
- در `script.ts` با `import` استفاده شود و اثر آن در صفحه **دیده شود**، نه فقط نصب شده باشد.
- Framework (مثل React، Vue و jQuery) و کتابخانه جایگزین `fetch` (مثل axios) قبول نیست.
- دو پکیج کار تکراری نکنند (مثلاً دو کتابخانه Toast).
- قبل از نصب در npmjs.com بررسی کنید: دانلود هفتگی، تاریخ آخرین انتشار، حجم و داشتن نوع TypeScript (علامت **TS** کنار نام پکیج).

چند پیشنهاد (می‌توانید پکیج دیگری انتخاب کنید):

- **`canvas-confetti`:** جشن کاغذرنگی وقتی کالا به سبد اضافه می‌شود
- **`notyf`:** پیام کوچک «به سبد اضافه شد»
- **`sweetalert2`:** پنجره تأیید «سبد خرید خالی شود؟»
- **`fuse.js`:** جستجوی هوشمند در نام کالاها، حتی با غلط تایپی
- **`embla-carousel`:** کاروسل با کشیدن انگشت روی موبایل
- **`tippy.js`:** Tooltip برای برچسب‌های کالا
- **`@formkit/auto-animate`:** حرکت نرم وقتی کارت‌ها اضافه یا حذف می‌شوند
- **`countup.js`:** شمارنده متحرک برای تعداد یا جمع سبد خرید

راهنمایی:

- **`TS7016: Could not find a declaration file for module 'canvas-confetti'`:** این پکیج نوع TypeScript ندارد و نوع آن جداگانه منتشر شده است. پکیج نوع را به‌عنوان ابزار نصب کنید: `npm install -D @types/canvas-confetti`
- **CSS پکیج:** بعضی پکیج‌ها فایل CSS دارند. آن را هم در `script.ts` `import` کنید؛ Vite آن را به صفحه اضافه می‌کند:

```
import { Notyf } from 'notyf'
import 'notyf/notyf.min.css'
```

- برای اعداد و تاریخ فارسی **پکیج لازم نیست**؛ خود مرورگر بلد است: `new Intl.NumberFormat('fa-IR')` و `new Intl.DateTimeFormat('fa-IR')`. همیشه اول بپرسید: «اصلاً به پکیج نیاز دارم؟»

نمونه: هر دو پکیج در کلیک دکمه افزودن:

```
import confetti from 'canvas-confetti'
notyf.success(button.dataset.name + ' به سبد اضافه شد')
confetti({ particleCount: 60, spread: 70 })
```

امتیاز اضافه: همه‌چیز نوع‌دار

۱. خط `"noImplicitAny": false` را از `tsconfig.json` پاک کنید و دوباره `npm run check` بگیرید:

```
script.ts(19,36): error TS7006: Parameter 'p' implicitly has an 'any' type.
```

۱. فقط یک خطا؟ مراقب باشید. `res.json()` مقدار `any` برمی‌گرداند و `any` بی‌صدا در همه کد پخش می‌شود. TypeScript دیگر روی آن داده‌ها هیچ بررسی‌ای انجام نمی‌دهد. پس صفر شدن خطاها کافی نیست:

۲. برای داده‌های API که استفاده می‌کنید `type` بنویسید (`Product`، `Category`، `Offer`، `Shop`...) و آن‌ها را از روی فایل‌های JSON بسازید.

۳. توابعی که داده می‌آورند نوع خروجی داشته باشند: `Promise<Product[]>`.

۴. همه پارامترهای توابع و مقدار بازگشتی نوع داشته باشند.

۵. همه متغیرها نوع داشته باشند، یا نوعشان از یک مقدار نوع‌دار معلوم باشد.

۶. در کل فایل هیچ `any` نوشته نشود. در `catch` از `unknown` استفاده کنید: `.catch (err: unknown)`

۷. برای جست‌وجو از نوع‌های عمومی (Generic) استفاده کنید: `Map<number, Offer>` و `Record<number, number>`.

نمونه:

```
type Product = {
  id: number
  categoryId: number
  name: string
  emoji: string
  unit: string
  price: number
  tags: string[]
}

const getProducts = async (): Promise<Product[]> => {
  const res: Response = await fetch('../../api/products.json')
  if (!res.ok) throw new Error('HTTP ' + res.status)
  return res.json()
}

let cart: number = Number(localStorage.getItem('kiss-cart') || 0)
const dairy: Product[] = products.filter((p: Product): boolean => p.categoryId === 2)
```

راهنمایی: نوع، یک **قول** به TypeScript است، نه بررسی در زمان اجرا. اگر سرور داده دیگری بفرستد، `type` جلوی آن را نمی‌گیرد. برای همین `res.ok` را همچنان بررسی کنید.

قوانین و تحویل

اجرا، Build و بررسی

```
npm run dev       # کار روزانه: صفحه با هر ذخیره خودش به‌روز می‌شود
npm run check     # فقط بررسی نوع‌ها
npm run build     # tsc + ساخت dist/
npm run preview   # اجرای dist/، شبیه سایت واقعی
```

هر چهار دستور را امتحان کنید:

- **`npm run dev`:** همه چیز مثل هفته دوم کار کند: کارت‌ها، حالت بارگذاری و خطا، سبد خرید و دو پکیج جدید.
- **`npm run build`:** اگر حتی یک خطای TypeScript وجود داشته باشد، Build ساخته نمی‌شود. بعد از Build داخل `dist/assets/` را ببینید: نام فایل‌ها یک Hash دارند.
- **`npm run preview`:** در DevTools → Network تعداد فایل‌های JavaScript را با `npm run dev` مقایسه کنید.

قوانین

- فقط فایل‌های پوشه `shops/<slug>/` خودتان تغییر کنند. پوشه `api/` و فایل‌های ریشه مخزن را تغییر ندهید.
- `node_modules/` و `dist/` Commit نشوند؛ `package.json` و `package-lock.json` حتماً Commit شوند.
- هیچ خطایی در Console و هیچ خطایی در `npm run check` دیده نشود.
- HTML صفحه همچنان باید از Validator پروژه با موفقیت عبور کند.

تحویل

یک Branch با نام `feat/<slug>-w3-ts` بسازید و یک Pull Request به مخزن Gang of Forty ارسال کنید.

توضیح Pull Request باید شامل موارد زیر باشد:

۱. **سطح:** فقط قبولی یا امتیاز اضافه؟ چک‌لیست Definition of Done را کپی کنید و موارد انجام‌شده را علامت بزنید.
۲. خروجی `npm run check` قبل از درست کردن خطاها (چند خط اول) و بعد از آن (خالی).
۳. **دو پکیج:** نام، نسخه، اینکه کجای صفحه استفاده شده‌اند و چرا آن‌ها را انتخاب کردید (دانلود هفتگی، حجم، نوع TypeScript). یک تصویر از صفحه که اثر هر دو پکیج در آن دیده شود.
۴. **یک خطای TypeScript** که برایتان جالب بود: چه می‌گفت و چطور درستش کردید؟
۵. تفاوت `dependencies` و `devDependencies` چیست و هر پکیج شما در کدام قرار گرفت؟
۶. چرا `package-lock.json` را Commit می‌کنیم ولی `node_modules/` را نه؟
۷. (امتیاز اضافه) چرا با پاک کردن `noImplicitAny` همه `any`ها پیدا نمی‌شوند؟

معیار انجام (Definition of Done)

سطح قبولی

- ☐ package.json با `"private": true`، `"type": "module"` و اسکریپت‌های dev، build، preview و check
- ☐ typescript و vite در devDependencies؛ package-lock.json Commit شده؛ node_modules/ و dist/ Commit نشده‌اند
- ☐ tsconfig.json با `"strict": true` و vite.config.ts در پوشه خودتان
- ☐ script.js حذف شده، script.ts وجود دارد و index.html آن را با `type="module"` بارگذاری می‌کند
- ☐ همه querySelectorها نوع دقیق دارند (`querySelector<HTMLUListElement>(…)`) و حالت null بررسی شده است
- ☐ همه عنصرهای ساخته‌شده با createElement و همه پارامترهای رویداد نوع دارند
- ☐ حداقل دو پکیج در dependencies که در script.ts استفاده شده‌اند و اثرشان در صفحه دیده می‌شود
- ☐ `npm run check` بدون خطا و `npm run build` موفق
- ☐ صفحه در `npm run dev` و `npm run preview` درست کار می‌کند و Console خطا ندارد
- ☐ بدون ! و as برای ساکت کردن خطا

امتیاز اضافه

- ☐ خط `"noImplicitAny": false` پاک شده و `npm run check` همچنان بدون خطاست
- ☐ برای داده‌های API type نوشته شده و توابع fetch نوع خروجی `Promise<…>` دارند
- ☐ همه متغیرها، پارامترها و مقدارهای بازگشتی نوع دارند
- ☐ هیچ any در کد نیست و catch از unknown استفاده می‌کند

چالش اختیاری: کد را به چند ماژول تقسیم کنید: types.ts (همه type‌ها)، api.ts (توابع fetch) و script.ts (DOM و رویدادها). از `import type { Product } from './types'` استفاده کنید. یا آدرس API را در فایل .env.development با VITE_API_URL قرار دهید و در کد از import.meta.env.VITE_API_URL بخوانید.